mini-scope – Simulator Guide

Testing firmware and user interface without hardware

October 22, 2026

1. What this is about

Without a board, mini-scope can run against the **MPLAB X simulator**. The real firmware from the test project `test.X`, compiled for the dsPIC33CK64MC103, runs in the simulator. The user interface looks the same as with hardware; only the port is called **MDB**.

How the parts fit together:

```
Browser (user interface)
   |
scope_gui.py  ──  tools/mdb_serial.py  ──  MDB (MPLAB X command line debugger, Java)
                                                  |
                                       simulator dsPIC33CK64MC103
                                       firmware test.X (configuration "sim")
```

- **PC → firmware:** the simulator's UART cannot receive. `mdb_serial.py` therefore writes the commands via MDB directly into a mailbox in RAM (`sim_rx`, `sim_rx_len` in `example_main.c`), where the firmware picks them up.
- **Firmware → PC:** whatever the firmware sends over its UART, the simulator writes to a file. `mdb_serial.py` reads that file as it grows.

2. Requirements

What	Where
MPLAB X v6.35	`C:\Program Files\Microchip\MPLABX\v6.35`
XC-DSC v3.30	`C:\Program Files\Microchip\xc-dsc\v3.30`
dsPIC33CK-MC_DFP 1.10.386	in the MPLAB X pack folder
Python 3.9 or later	the first start sets up everything else

For other versions, adjust the paths at the top of **`tools\mdb_serial.py`** (`MPLABX`, `XC_DSC`, `DFP`) and in **`start_sim.bat`** (`MPLABX`).

3. Starting

Double-click **`start_sim.bat`** in the `mini-scope` folder.

The batch file does three things:

1. On the first start it creates the Python environment `.venv` and installs the packages (about a minute).
2. It builds `test.X` in configuration **sim**, but only if something has changed. On an error the window stays open and the reason is in `test.X\build_sim.log`.
3. It starts the user interface and opens the browser at http://localhost:8080.

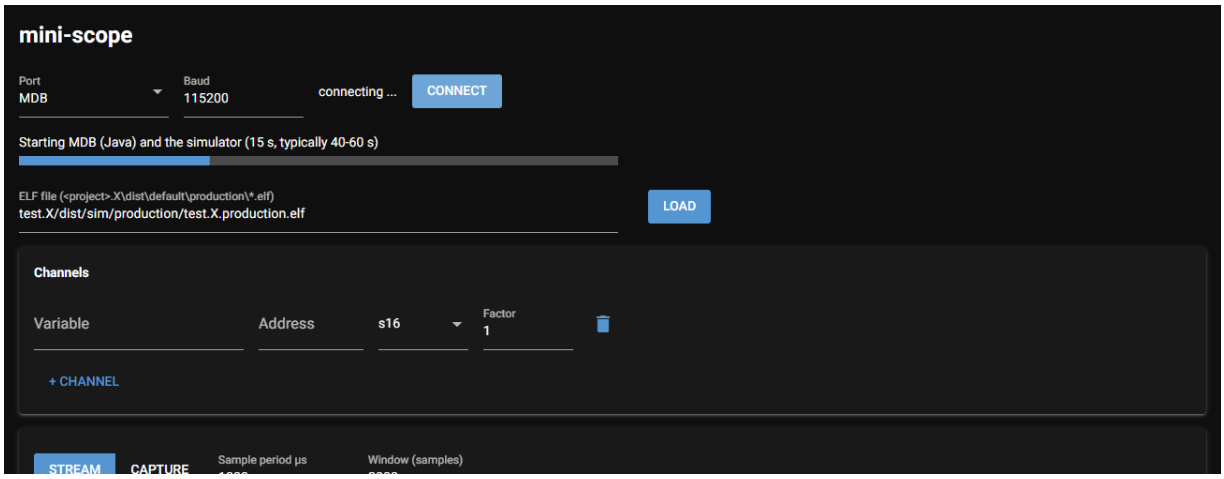
Keep the console window open while you work.

4. Connecting

1. **Port** already shows **MDB**. Click **Connect**.
2. A progress bar shows what is happening:

Display	What happens	Duration
Starting MDB (Java) and the simulator	Java and MDB start up	about 30 s
Simulator ready, loading firmware	the ELF is loaded into the simulator	about 5 s
Firmware starting, waiting for the first answer	first command to the firmware	about 5–10 s
Firmware responds, waiting for the last bytes	cleanup	about 6 s

Then the bar disappears and the status line shows **“connected, target responds”**. Altogether this takes 40–60 s.



Progress bar during start-up

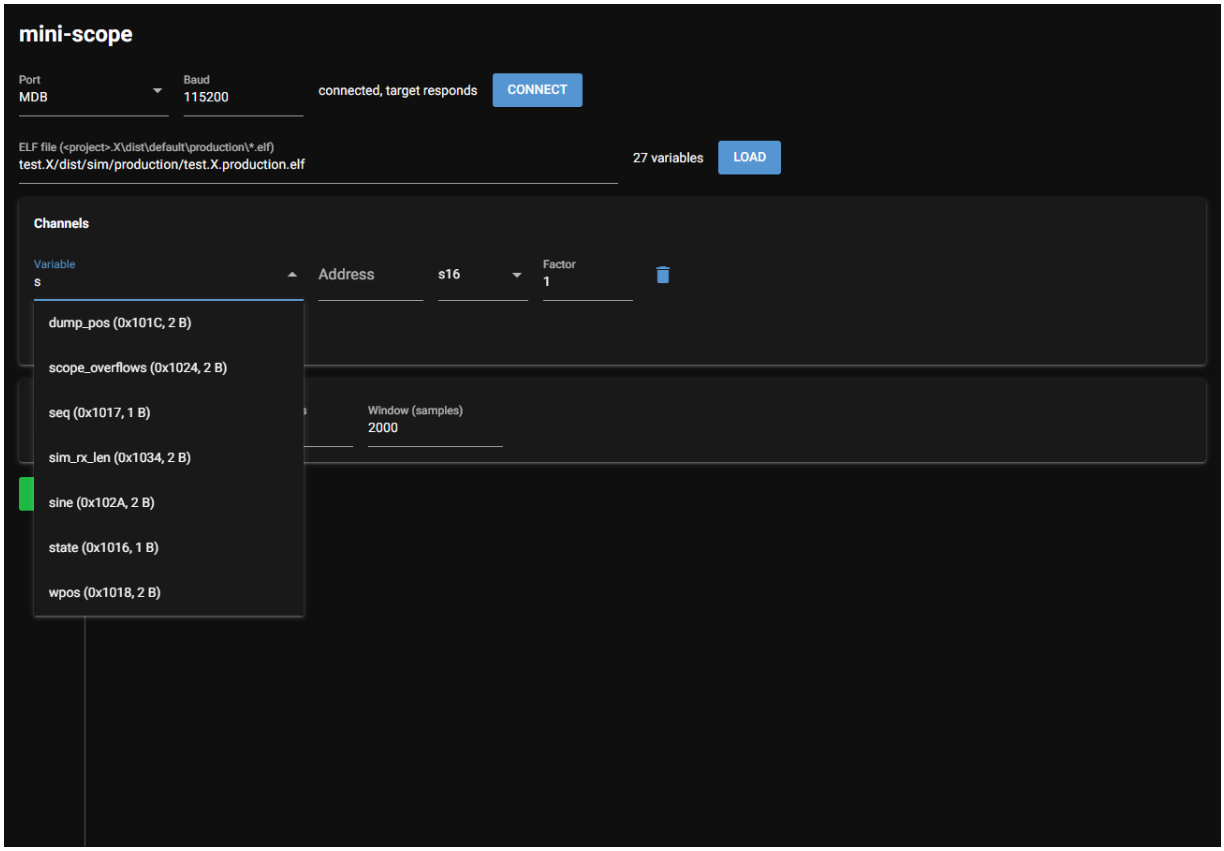
5. Measuring

1. Click **Load**. The ELF of the simulator build is already filled in.
2. Click the **Variable** field and pick a variable from the list.
3. Add more variables with **+ Channel**.

The demo signals from `example_main.c` are a good start:

Variable	Type	Shape
`sine`	s16	sine, ±32767
`ramp`	u16	sawtooth 0 … 65535 (shown as `s16` it jumps at 32767)
`counter`	s32	counts every pass of the main loop
`toggle`	u8	alternates between 0 and 1

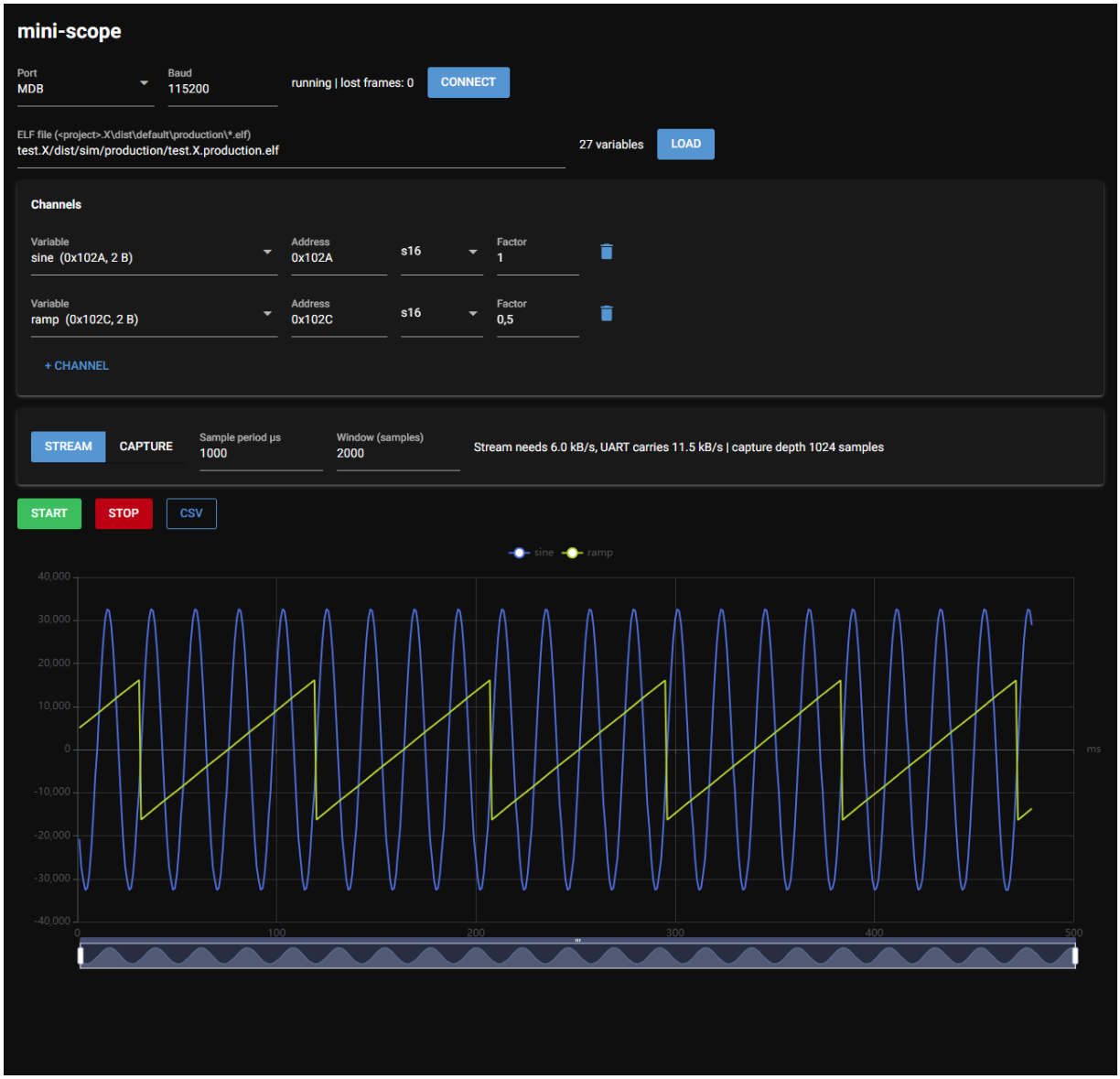
The list also contains the internal variables of `scope.c` (`head`, `tail`, `state` …). You normally do not need them.



Picking variables

Stream

Select **Stream**, set **Sample period µs** to 1000 and click **Start**. The curve appears after a few seconds.

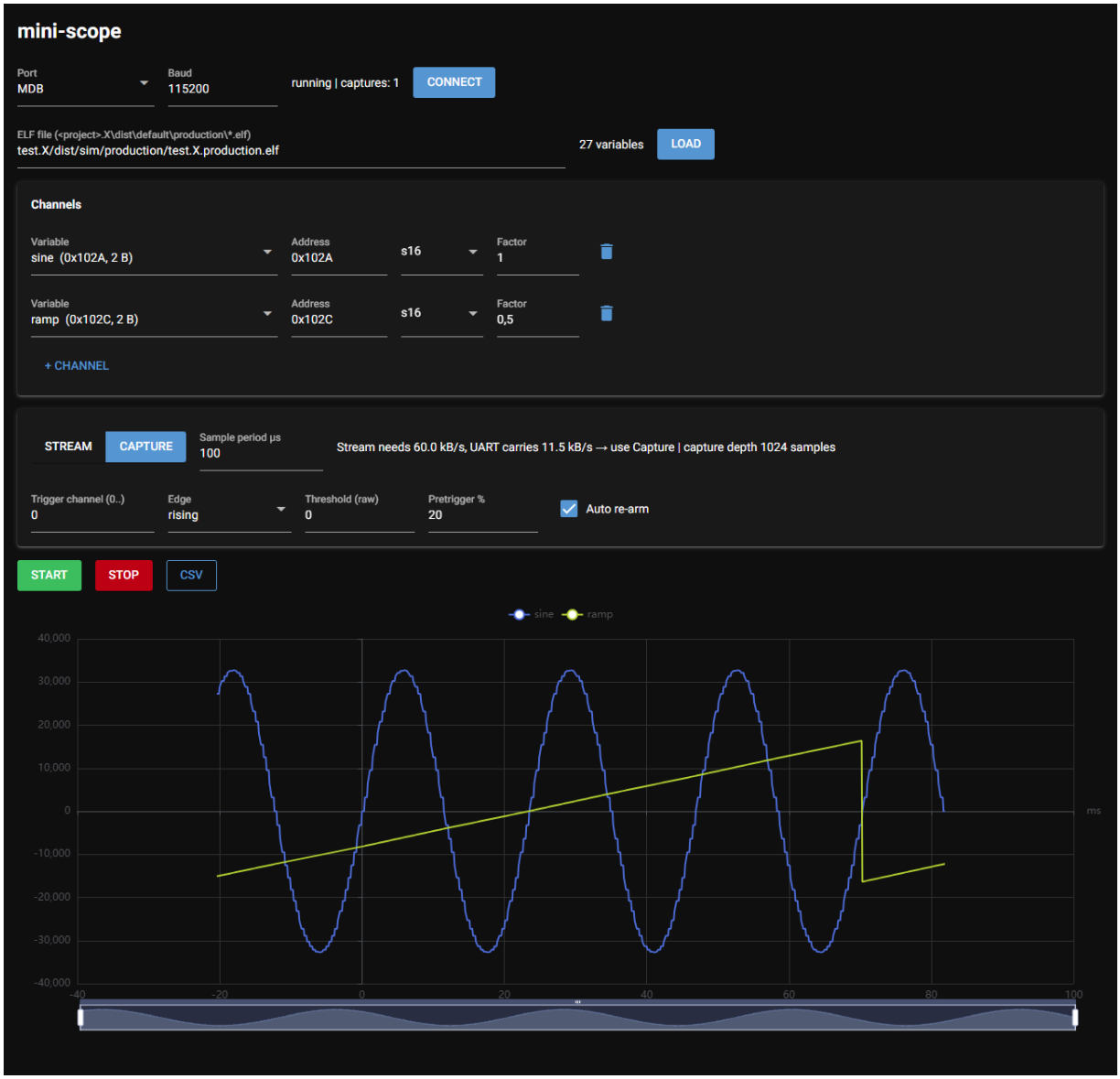


Stream in the simulator: sine and ramp

Capture

1. Click **Stop**.
2. Select **Capture** and set the sample period to 100 µs.
3. Trigger channel 0 (`sine`), edge **rising**, threshold 0.
4. Uncheck **Auto re-arm** if you want a single recording.
5. Click **Start**.

The recording appears after 10–30 s; the trigger is at 0 ms.



Capture in the simulator

6. What is different in the simulator

- **Speed:** the simulator runs about 25 times slower than real time, so 1 s of simulated time takes about 25 s. The time axis in the user interface shows the **simulated** time and is therefore correct.
- **Delay:** data appears a few seconds late, because the simulator writes the UART file in blocks.
- **Stop takes time:** after **Stop** the user interface waits about 6 s until no late bytes arrive.
- **Reset: Reset** restarts the firmware with the `reset` instruction, as on the target. If the firmware no longer answers, the user interface resets the simulated device through MDB instead. Allow up to 40 s.
- **No hardware effects:** noise, timing on real pins and the behaviour of the PICkit cannot be checked in the simulator. It only shows whether the firmware logic is right.

7. Testing your own code in the simulator

1. Add your own variables and code to `firmware\example_main.c`, or add your own `.c` files to `test.X`.
2. **Do not remove** the mailbox polling in the main loop (`#ifdef __MPLAB_DEBUGGER_SIMULATOR`). It must run regularly, otherwise no commands arrive.
3. Put waits on hardware flags that the simulator never sets (PLL lock, oscillator switch) inside `#ifndef __MPLAB_DEBUGGER_SIMULATOR`. `example_main.c` shows how.
4. Close the console window and run `start_sim.bat` again. It rebuilds automatically.
5. After connecting, click **Load** again so that the new addresses apply.

The test project can also be opened in MPLAB X (`test.X`). Select configuration **sim** to debug there with the IDE simulator, or **default** to program real hardware with the PICkit 5.

8. Quitting

Click **Stop**, then close the console window or press Ctrl+C. This also ends the simulator.

If the user interface crashed, an MDB process may be left in the background. It shows up in Task Manager as **java.exe** with high memory use. End it in PowerShell like this:

```
Get-CimInstance Win32_Process -Filter "Name='java.exe'" |
  Where-Object { $_.CommandLine -like '*mdb.jar*' } |
  ForEach-Object { Stop-Process -Id $_.ProcessId -Force }
```

The MPLAB X IDE itself is not affected; its Java process has no `mdb.jar` in its command line.

9. Troubleshooting

Symptom	Cause and remedy
Window says “Build failed”	error in the firmware: see `test.X\build_sim.log`
Port **MDB** missing from the list	the user interface was not started with `start_sim.bat`
Progress stalls for more than 90 s, then “Error”	check the paths in `tools\mdb_serial.py`. An old MDB process may block; end it as in section 8 and start again.
“MDB: firmware does not respond”	the mailbox polling in the main loop is missing, or the firmware hangs in a wait loop (see section 7, step 3)
Curve stays empty, status “running”	wait a few seconds. If nothing comes, **Load** the ELF again and reselect the channels.
No capture appears	the threshold is never crossed: try edge **none**
Page stops responding	press F5 and click **Connect** again. This starts a new simulator; the old one ends when the user interface is closed.

10. Fast alternative: PC model

For quick tests of the user interface there is also a model that runs `scope.c` directly on the PC, without MPLAB X. It runs in real time and needs no start-up time, but it does not check the compiled dsPIC firmware.

```
gcc -O2 -shared -Itests/mock -Ifirmware -o tests/sim.dll tests/mock/sim.c firmware/scope.c
start_gui.bat --sim
```

Select port **SIM**, click **Connect**, leave the ELF field empty and click **Load**. `gcc` is available for example through MinGW. The DLL only needs rebuilding after changes to `scope.c`.